

```

"""
train_model.py
-----
Trains a Convolutional Neural Network to classify
facial expressions
using the FER-2013 dataset (fer2013.csv).

Dataset:
    Download 'fer2013.csv' from Kaggle:
    https://www.kaggle.com/datasets/msambare/fer2013
    (or
    https://www.kaggle.com/datasets/deadskull7/fer2013)
    Place it in the `data/` folder before running this
    script.

Usage:
    python train_model.py
"""

import os
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, Dropout, Flatten, Dense,
    BatchNormalization
)
from tensorflow.keras.callbacks import
ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.preprocessing.image import
ImageDataGenerator

# -----
# -----
# Config
# -----

```

```

-----
DATA_PATH = os.path.join("data", "fer2013.csv")
MODEL_OUT = os.path.join("model", "emotion_model.h5")
IMG_SIZE = 48
NUM_CLASSES = 7
BATCH_SIZE = 64
EPOCHS = 60

EMOTIONS = {
    0: "Angry", 1: "Disgust", 2: "Fear", 3: "Happy",
    4: "Sad", 5: "Surprise", 6: "Neutral"
}

def load_data(path):
    """Parses fer2013.csv into image arrays and labels
    split by usage."""
    df = pd.read_csv(path)

    pixels = df["pixels"].apply(
        lambda p: np.array(p.split(),
dtype="float32").reshape(IMG_SIZE, IMG_SIZE, 1)
    )
    X = np.stack(pixels.values) / 255.0
    y = to_categorical(df["emotion"].values,
NUM_CLASSES)

    train_mask = df["Usage"] == "Training"
    val_mask = df["Usage"] == "PublicTest"
    test_mask = df["Usage"] == "PrivateTest"

    return (
        X[train_mask], y[train_mask],
        X[val_mask], y[val_mask],
        X[test_mask], y[test_mask],
    )

def build_model():
    """Defines a compact CNN suitable for 48x48
    grayscale emotion classification."""
    model = Sequential([

```

```

        Conv2D(64, (3, 3), activation="relu",
padding="same", input_shape=(IMG_SIZE, IMG_SIZE, 1)),
        BatchNormalization(),
        Conv2D(64, (3, 3), activation="relu",
padding="same"),
        BatchNormalization(),
        MaxPooling2D(2, 2),
        Dropout(0.25),

        Conv2D(128, (3, 3), activation="relu",
padding="same"),
        BatchNormalization(),
        Conv2D(128, (3, 3), activation="relu",
padding="same"),
        BatchNormalization(),
        MaxPooling2D(2, 2),
        Dropout(0.25),

        Conv2D(256, (3, 3), activation="relu",
padding="same"),
        BatchNormalization(),
        MaxPooling2D(2, 2),
        Dropout(0.25),

        Flatten(),
        Dense(256, activation="relu"),
        BatchNormalization(),
        Dropout(0.5),
        Dense(NUM_CLASSES, activation="softmax"),
    ])

    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

def main():
    if not os.path.exists(DATA_PATH):
        raise FileNotFoundError(

```

```

        f"'{DATA_PATH}' not found. Download
fer2013.csv from Kaggle and "
        f"place it under the 'data/' folder."
    )

    os.makedirs("model", exist_ok=True)

    print("Loading dataset...")
    X_train, y_train, X_val, y_val, X_test, y_test =
load_data(DATA_PATH)
    print(f"Train: {X_train.shape}, Val:
{X_val.shape}, Test: {X_test.shape}")

    datagen = ImageDataGenerator(
        rotation_range=10,
        zoom_range=0.1,
        width_shift_range=0.1,
        height_shift_range=0.1,
        horizontal_flip=True,
    )
    datagen.fit(X_train)

    model = build_model()
    model.summary()

    callbacks = [
        ModelCheckpoint(MODEL_OUT,
monitor="val_accuracy", save_best_only=True,
verbose=1),
        EarlyStopping(monitor="val_accuracy",
patience=10, restore_best_weights=True),
        ReduceLROnPlateau(monitor="val_loss",
factor=0.5, patience=5, min_lr=1e-6),
    ]

    model.fit(
        datagen.flow(X_train, y_train,
batch_size=BATCH_SIZE),
        validation_data=(X_val, y_val),
        epochs=EPOCHS,
        callbacks=callbacks,
    )

```

```
        test_loss, test_acc = model.evaluate(X_test,
y_test, verbose=0)
        print(f"Test accuracy: {test_acc * 100:.2f}%")

        model.save(MODEL_OUT)
        print(f"Model saved to {MODEL_OUT}")

if __name__ == "__main__":
    main()
```